Lecture 9

Introduction to VHDL Project

Important Notes

- **VHDL Project – Matrix Processing Core Design**
  - D = A x B x C, where A, B and C are of size 8 x 16, 16 x 16 and 16 x 8, respectively
  - 8-bit signed integer for the elements of all the input matrix
- **Reference Design – Matrix Multiplier Core Design**
  - 4 x 4
  - 8-bit signed integer
- **Submission**
  - Moodle submission (See detail submission requirements in the project specification)
  - Deadline – **<u>3:00pm, Tuesday, 5th May 2026</u>**
- **Late submission**
  - A penalty of 5% per working day will be imposed for the late submission.

Project Marking Scheme

- **Design, Implementation and Verification (60 marks):**
  - Matlab model (5 marks)
    - Signed integer data
    - Able to automatically generate all the test files
  - VHDL code compilation (5 marks)
    - No compilation errors
  - Synthesis (15 marks)
    - No synthesis errors, such as multi-drive errors
    - No critical warnings
    - Allow non-critical warnings, which cannot be ignored, but 5 marks' penalty per non-critical warning.
  - Behavioural simulation (5 marks)
    - Behavioural simulation can be launched without any errors

Project Marking Scheme

- **Design, Implementation and Verification (60 marks):**
  - Correct data (10 marks)
    - The result data from VHDL simulation must match that generated from your Matlab model
  - Design efficiency (10 marks)
    - It is assessed, in terms of the number of the clock cycles from the rising edge of WriteEnable to the rising edge of DataReady
  - Coding quality and readability (10 marks)
    - Must use one state machine with two processes, one of which is a clocked process and the other is purely combinational process
    - Concise code comments

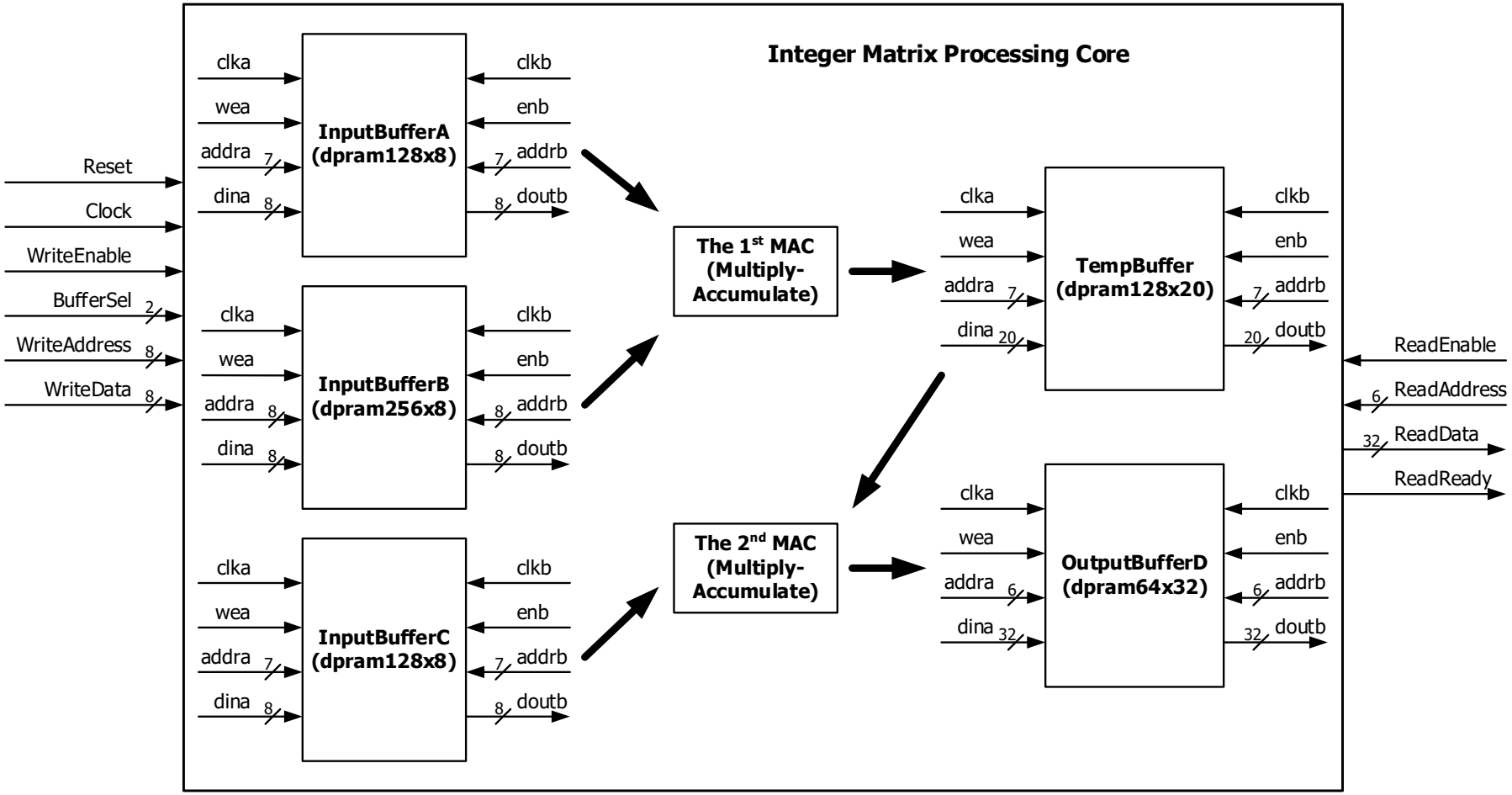
Project Marking Scheme

- **10-page report (40 marks):**
  - Description of project design (10 marks)
    - A state transition diagram must be included
  - Simulation and discussion (10 marks)
    - Including simulation waveforms to show those key internal signals to justify your verification process
  - Conclusions (6 marks)
  - Further work (4 marks)
  - Report quality (10 marks)
    - Clarity
    - Appropriate format and structure
    - Length

Matrix Multiplication and Addition

$$
\begin{array}{ccccccccc}
\mathbf{A} & & \mathbf{B} & & \mathbf{C} & & \mathbf{D} \\
\begin{bmatrix}
a11 & a12 & a13 & \cdots & a116 \\
a21 & a22 & a23 & \cdots & a216 \\
a31 & a32 & a33 & \cdots & a316 \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
a81 & a82 & a83 & \cdots & a816
\end{bmatrix}
& \mathbf{X} &
\begin{bmatrix}
b11 & b12 & b13 & \cdots & b116 \\
b21 & b22 & b23 & \cdots & b216 \\
b31 & b32 & b33 & \cdots & b316 \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
b161 & b162 & b163 & \cdots & b1616
\end{bmatrix}
& \mathbf{X} &
\begin{bmatrix}
c11 & c12 & c13 & \cdots & c18 \\
c21 & c22 & c23 & \cdots & c28 \\
c31 & c32 & c33 & \cdots & c38 \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
c161 & c162 & c163 & \cdots & c168
\end{bmatrix}
& = &
\begin{bmatrix}
d11 & d12 & d13 & \cdots & d18 \\
d21 & d22 & d23 & \cdots & d28 \\
d31 & d32 & d33 & \cdots & d38 \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
d81 & d82 & d83 & \cdots & d88
\end{bmatrix}
\end{array}
$$

Matrix Processing Core



Buffer Data Organisation

InputBufferA (dpram128x8)

0	a11
1	a12
2	a13
⋮	
15	a116
16	a21
17	a22
18	a23
⋮	
31	a216
32	a31
33	a32
34	a33
⋮	
47	a316
⋮	
112	a81
113	a82
114	a83
⋮	
127	a816

InputBufferB (dpram256x8)

0	b11
1	b12
2	b13
⋮	
15	b116
16	b21
17	b22
18	b23
⋮	
31	b216
32	b31
33	b32
34	b33
⋮	
47	b316
⋮	
240	b161
251	b162
252	b163
⋮	
255	b1616

InputBufferC (dpram128x8)

0	c11
1	c12
2	c13
⋮	
8	c18
9	c21
10	c22
11	c23
⋮	
15	c28
16	c31
17	c32
18	c33
⋮	
23	c38
⋮	
120	c161
121	c162
122	c163
⋮	
127	c168

TempBuffer (dpram128x20)

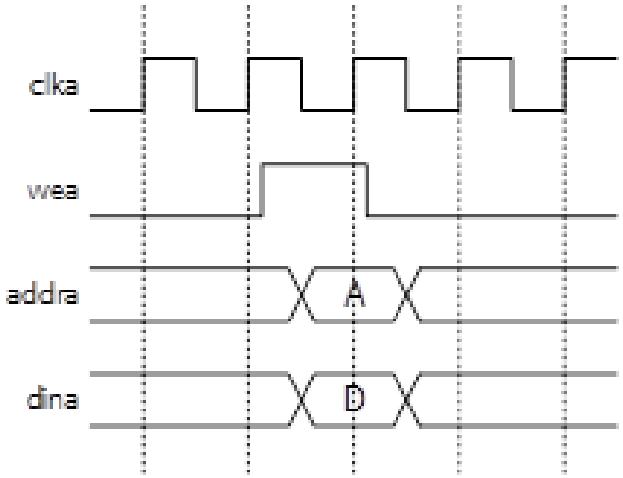
0	t11
1	t12
2	t13
⋮	
15	t116
16	t21
17	t22
18	t23
⋮	
31	t216
32	t31
33	t32
34	t33
⋮	
47	t316
⋮	
112	t81
113	t82
114	t83
⋮	
127	t816

OutputBufferD (dpram64x32)

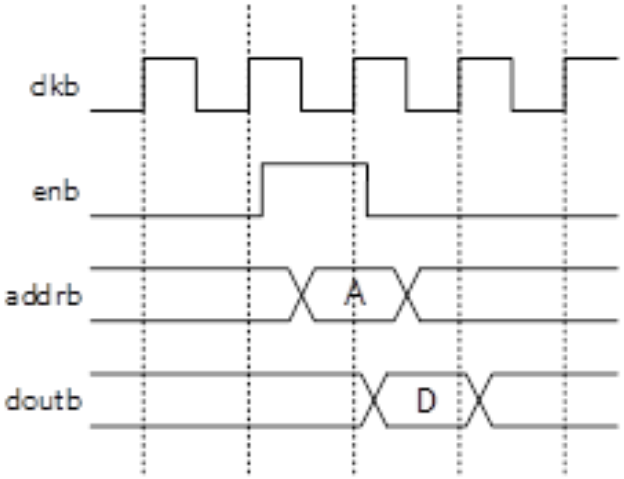
0	d11
1	d12
2	d13
⋮	
7	d18
8	d21
9	d22
10	d23
⋮	
15	d28
16	d31
17	d32
18	d33
⋮	
23	d38
⋮	
56	d81
57	d82
58	d83
⋮	
63	d88

Timing for Write to/Read from DPRAM

- **Write Operation**



- **Read Operation**



Referenece Design

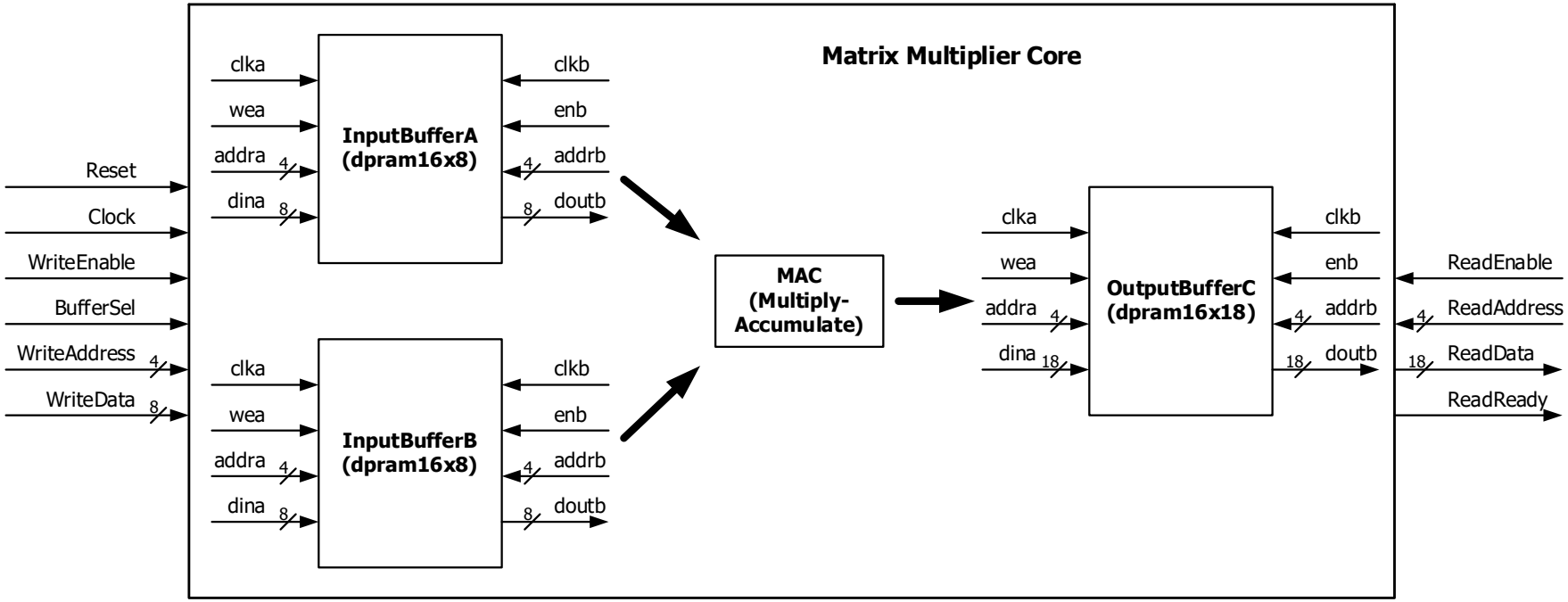
- **Matrix Multiplication**
  - Size of 4 x 4, 8 bits for each matrix elements of the input matrixes

$$\overset{A}{\begin{bmatrix} a11 & a12 & a13 & a14 \\ a21 & a22 & a23 & a24 \\ a31 & a32 & a33 & a34 \\ a41 & a42 & a43 & a44 \end{bmatrix}} \times \overset{B}{\begin{bmatrix} b11 & b12 & b13 & b14 \\ b21 & b22 & b23 & b24 \\ b31 & b32 & b33 & b34 \\ b41 & b42 & b43 & b44 \end{bmatrix}} = \overset{C}{\begin{bmatrix} c11 & c12 & c13 & c14 \\ c21 & c22 & c23 & c24 \\ c31 & c32 & c33 & c34 \\ c41 & c42 & c43 & c44 \end{bmatrix}}$$

$Cij = ai1 \times b1j + ai2 \times b2j + ai3 \times b3j + ai4 \times b4j;\ i = 1, 2, 3, 4;\ j = 1, 2, 3, 4$

Referenece Design

- **Matrix Multiplier Core Design**



Referenece Design

- **Buffer Organisation**

	InputBufferA (dpram16x8)
0	a11
1	a12
2	a13
3	a14
4	a21
5	a22
6	a23
7	a24
8	a31
9	a32
10	a33
11	a34
12	a41
13	a42
14	a43
15	a44

	InputBufferB (dpram16x8)
0	b11
1	b12
2	b13
3	b14
4	b21
5	b22
6	b23
7	b24
8	b31
9	b32
10	b33
11	b34
12	b41
13	b42
14	b43
15	b44

	OutputBufferC (dpram16x18)
0	c11
1	c12
2	c13
3	c14
4	c21
5	c22
6	c23
7	c24
8	c31
9	c32
10	c33
11	c34
12	c41
13	c42
14	c43
15	c44

IP Catalog to Generate Dual Port RAM

IntMatrixMulCore_4x4_8bits - [C:/VHDL/IntMatrixMulCore_4x4_8bits/IntMatrixMulCore_4x4_8bits/IntMatrixMulCore_4x4_8bits.xpr] - Vivado 2018.3

File Edit Flow Tools Reports Window Layout View Help Quick Access

Flow Navigator

PROJECT MANAGER
- Settings
- Add Sources
- Language Templates
- IP Catalog

IP INTEGRATOR
- Create Block Design
- Open Block Design
- Generate Block Design

SIMULATION
- Run Simulation

PROJECT MANAGER - IntMatrixMulCore_4x4_8bits

Sources
- Design Sources (1)
  - IntMatMulCore(IntMatMulCo
- Constraints
- Simulation Sources (4)
- Utility Sources

Hierarchy | IP Sources | Li

IP Properties
Block Memory Generator

Project Summary | IP Catalog

Cores | Interfaces

Search:

Name	AXI4
Debug & Verification	
Memories & Storage Elements	
FIFOs	
ECC	
Memory Interface Generators	
RAMs & ROMs	
RAMs & ROMs & BRAM	
Block Memory Generator	AXI4
Basic Elements	

Dual Port RAM Generation

Project Summary | IP Catalog

Cores | Interfaces

Search:

Name	AXI4	Status	License	VLNV
Memories & Storage Elements				
ECC		Production	Included	xilinx.com:ip:ecc:2.0
FIFOs				
Memory Interface Generators				
RAMs & ROMs				
RAMs & ROMs & BRAM				
Block Memory Generator	AXI4	Production	Included	xilinx.com:ip:blk_mem_gen:8.4

Details

Name: **Block Memory Generator**

Version: 8.4 (Rev. 2)

Interfaces: AXI4

Description: The Xilinx LogiCORE IP Block Memory Generator replaces the Dual Port Block Memory and Single Port Block Memory LogiC

Dual Port RAM Generation (Basic)

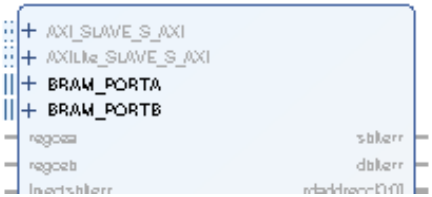
Re-customize IP

Block Memory Generator (8.4)

Documentation | IP Location | Switch to Defaults

IP Symbol | Power Estimation

☑ Show disabled ports



Component Name: dpram16x8

Basic | Port A Options | Port B Options | Other Options | Summary

Interface Type: Native

☐ Generate address interface wi

Memory Type: Simple Dual Port RAM

☐ Common Clock

ECC Options

ECC Type: No ECC

☐ Error Injection Pins: Single Bit Error Injection

Write Enable

☐ Byte Write Enable

Byte Size (bits): 9

Dual Port RAM Generation (Basic)

Re-customize IP

Block Memory Generator (8.4)

Documentation | IP Location | Switch to Defaults

IP Symbol | Power Estimation

Show disabled ports

AXI_SLAVE_S_AXI
AXILite_SLAVE_S_AXI
BRAM_PORTA
BRAM_PORTB
regcea
regceb
injectsbiterr
injectdbiterr
sbiterr
dbiterr
rdaddrecc[31:0]
rsta_busy

Component Name: dpram16x8

Basic | Port A Options | Port B Options | Other Options | Summary

Memory Size

Port A Width: 8 — Range: 1 to 4608 (bits)

Port A Depth: 16 — Range: 2 to 1048576

The Width and Depth values are used for Write Operations in Port A

Operating Mode: No Chan...

Enable Port Type: Always Enabled

Port A Optional Output Registers

Primitives Output Register | Core Output Register

SoftECC Input Register | REGCEA Pin

READ Address Change A

Dual Port RAM Generation (Basic)

Re-customize IP

Block Memory Generator (8.4)

Documentation | IP Location | Switch to Defaults

IP Symbol | Power Estimation

☑ Show disabled ports

AXI_SLAVE_S_AXI
AXILite_SLAVE_S_AXI
BRAM_PORTA
BRAM_PORTB
regcea
regceb
injectsbiterr
injectdbiterr
sbiterr
dbiterr
rdaddrecc[31:0]
rsta_busy

Component Name: dpram16x8

Basic | **Port A Options** | Port B Options | Other Options | Summary

Memory Size

Port A Width: 8 — Range: 1 to 4608 (bits)

Port A Depth: 16 — Range: 2 to 1048576

The Width and Depth values are used for Write Operations in Port A

Operating Mode: No Chan... — Enable Port Type: Always Enabled

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

READ Address Change A

Dual Port RAM Generation (Basic)

Re-customize IP

Block Memory Generator (8.4)

Documentation | IP Location | Switch to Defaults

IP Symbol | Power Estimation

☑ Show disabled ports

Component Name: dpram16x8

Basic | Port A Options | **Port B Options** | Other Options | Summary

Memory Size

Port B Width: 8

Port B Depth : 16

The Width and Depth values are used for Read Operation in Port B

Operating Mode: Write First

Enable Port Type: Use ENB Pin

Port B Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Output Register ☐ REGCEB Pin

Port B Output Reset Options

Dual Port RAM Generation (.VHO)

Windows (C:) › VHDL › IntMatrixMulCore_4x4_8bits › IntMatrixMulCore_4x4_8bits.ip_user_files › ip › dpram16x8

Name	Date modified	Type	Size
dpram16x8.veo	11/03/2026 13:44	VEO File	4 KB
dpram16x8.vho	11/03/2026 13:44	VHO File	4 KB
dpram16x8_stub.v	03/03/2026 11:16	V File	2 KB
dpram16x8_stub.vhdl	03/03/2026 11:16	VHDL File	2 KB

Dual Port RAM Generation (.VHO)

```
-- The following code must appear in the VHDL architecture header.

------------- Begin Cut here for COMPONENT Declaration ------ COMP_TAG
COMPONENT dpram16x8
  PORT (
    clka : IN STD_LOGIC;
    wea : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    addra : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
    dina : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
    clkb : IN STD_LOGIC;
    enb : IN STD_LOGIC;
    addrb : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
    doutb : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
  );
END COMPONENT;
-- COMP_TAG_END ------ End COMPONENT Declaration ------------

-- The following code must appear in the VHDL architecture
-- body. Substitute your own instance name and net names.

------------- Begin Cut here for INSTANTIATION Template ----- INST_TAG
your_instance_name : dpram16x8
  PORT MAP (
    clka => clka,
    wea => wea,
    addra => addra,
    dina => dina,
    clkb => clkb,
    enb => enb,
    addrb => addrb,
    doutb => doutb
  );
-- INST_TAG_END ------ End INSTANTIATION Template ---------
```

Matlab Code to Generate Test Data

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

RandomMatrixMulTest_4x4_8bits.m

```
% Clear variables and console
clear;
clc;

% Randomly generate 4x4 matrices A and B
% with values between -9 and 9
A = randi([-9 9], 4, 4);
B = randi([-9 9], 4, 4);

% Calculate C
C = A*B;

% Convert matrices to hexadecimal arrays
AHex = dec2hex(A', 2);
BHex = dec2hex(B', 2);
CHex = dec2hex(C', 5);


% Specify output file names
fileNameA =   'InputA.txt';
fileNameB =   'InputB.txt';
fileNameC =   'OutputC_matlab.txt';

% Output hexadecimal arrays representing each matrix
% to .txt files
fid = fopen(fileNameA, 'w');
fprintf(fid, '%s\n', string(AHex));
fclose(fid);
disp(['File: ' fileNameA ' has been saved.']);

fid = fopen(fileNameB, 'w');
fprintf(fid, '%s\n', string(BHex));
fclose(fid);
disp(['File: ' fileNameB ' has been saved.']);

fid = fopen(fileNameC, 'w');
fprintf(fid, '%s\n', string(CHex));
fclose(fid);
disp(['File: ' fileNameC ' has been saved.']);
```

MATrix LABoratory length : 93

Test Data from Matlab

D:\OneDrive - The University of N

File Edit Search View Encodi

InputA.txt

1	06
2	03
3	09
4	09
5	08
6	F8
7	09
8	00
9	F9
10	FC
11	F9
12	06
13	08
14	01
15	09
16	F9

D:\OneDrive - The University of No

File Edit Search View Encodin

InputB.txt

1	FF
2	03
3	03
4	03
5	08
6	F7
7	05
8	FA
9	06
10	07
11	05
12	04
13	09
14	08
15	FE
16	F7

D:\OneDrive - The University of Nottingham\Teacl

File Edit Search View Encoding Language

OutputC_matlab.txt

1	00099
2	0007E
3	0003C
4	FFFD3
5	FFFEE
6	0009F
7	0001D
8	0006C
9	FFFF3
10	0000E
11	FFFA8
12	FFFB1
13	FFFF7
14	00016
15	00058
16	00075

Result Data

C:\Temp\IntMatrixMulCore_4x4_8bits\IntMatrixMulCore_4x4_8bits.sim\sim_1\behav\xsim\OutputC.txt - Notepad++

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

OutputC.txt

```
Results
Data from Matlab     Data from Simulation
00099                00099                TRUE
0007E                0007E                TRUE
0003C                0003C                TRUE
FFFD3                FFFD3                TRUE
FFFEE                FFFEE                TRUE
0009F                0009F                TRUE
0001D                0001D                TRUE
0006C                0006C                TRUE
FFFF3                FFFF3                TRUE
0000E                0000E                TRUE
FFFA8                FFFA8                TRUE
FFFB1                FFFB1                TRUE
FFFF7                FFFF7                TRUE
00016                00016                TRUE
00058                00058                TRUE
00075                00075                TRUE
```